

Functions- A **function** is a block of code that performs a specific task. Instead of writing the same code multiple times, we write it once inside a function and call it whenever needed.

Advantages: Reusable code, Easy to understand , Reduces repetition

Types of Functions

- Built-in Functions (print(), len(), input())
- User-defined Functions

Function with Parameters :Parameters are values passed into a function.

Function Call:Calling a function means executing it.

Return Statement: The **return** statement sends a value back from a function. Once `return` is executed, the function stops.

- A function can return one or more values.
 - After `return`, the function stops executing.
- If there is no return statement, Python returns `None`.

Iterators

An **iterator** is an object that allows you to access one element at a time from a collection.

Instead of giving all values at once, it gives the next value whenever you ask for it.

- An iterator gives one element at a time.
- Created using `iter()`.
- Accessed using `next()`.

Generators:

A **generator** creates and returns data one item at a time using `yield`, which makes it more memory-efficient for large datasets.

- A generator is a special function.
- Uses the `yield` keyword.
- Produces one value at a time.

Lambda Function

A **lambda function** is a small, unnamed function used for simple operations.

Instead of writing a complete function using `def`, we can write it in **one line** using `lambda`.

Example:

```
square = lambda x: x * x
print(square(5))
```

`map()`: It applies the **same function** to **every element** in an iterable (list, tuple, etc.).
It **modifies every element** and returns a new iterable.

Syntax:**map(function, iterable)****Example:**

```
numbers = [1, 2, 3, 4]
result = map(lambda x: x * 2, numbers)
print(list(result))
```

filter():

`filter()` selects only the elements that satisfy a condition.

The function passed to `filter()` must return **True** or **False**.

Example:

```
numbers = [1, 2, 3, 4, 5, 6]

result = filter(lambda x: x % 2 == 0, numbers)

print(list(result))
```

reduce():

Use `reduce()` when you want to combine all values into one result.

Example:

```
from functools import reduce

numbers = [1, 2, 3, 4]

result = reduce(lambda x, y: x + y, numbers)

print(result)
```

output:10